

The Adherence Gap: Retrieval Is Not the Bottleneck for Graph-Augmented Coding Agents

Vitor Calvi · Kaggle: Google — The Gemma 4 Developer Agent Paper Track · 30 September 2026 Target award: Best New Application (\$10,000)

Abstract

Code-graph retrieval is the standard answer to repository-level coding-agent navigation, and the Gemma 4 Developer Agent competition ships it in production: `get_code_neighbors`, `search_similar_code`, and `get_code_subgraph`, backed by a per-repository graph and node embeddings. We test whether it works by joining two literatures usually reported separately: **localization research**, which measures whether the right function is in the candidate set, and **agent research**, which measures whether the task gets resolved. Joining them on 636 public end-to-end agent runs produces an uncomfortable answer. Editing the correct function is *necessary* for resolution — $P(\text{resolved} \mid \text{edited gold function}) = 0.568$ versus 0.069 otherwise, a risk ratio of **8.26×** (95% CI [5.55, 12.28], Fisher’s exact $p = 1.17\text{e-}45$). Yet in **46.6%** of runs where the retriever’s top-15 demonstrably *contained* the correct function, the agent still did not edit it. We call this the **adherence gap**, the binding constraint, not retrieval recall. A retrieval skill that prints a ranked list leaves resolve rate statistically unchanged (29.9% vs 28.9%; paired McNemar $p = 0.78$), because printing candidates does not change which file the agent opens. We derive a **commitment protocol** — encoded as native prompt scaffolding and tool-use schemas for Google ADK agents powered by Gemma 4 — and release the analysis code, the taxonomy, and a re-runnable script over the public logs.

1. Introduction

A repository-scale coding agent gets three navigation tools, a code graph, and pre-computed node embeddings. It must still decide *which file to open first*. The field’s implicit model is that better retrieval produces better patches.

We test that model and find it does not hold here, for a reason that is easy to miss. Localization papers report **accuracy@k** — is the right function in the top k? Agent papers report **resolve rate** — did the task get fixed? These are joined by an unexamined step: the agent *acting* on what it retrieved. A candidate list only helps if the agent follows it.

Our contribution is to make that step measurable and then to close it:

1. the **8.26× risk ratio** that quantifies the necessity of editing the correct function;
2. the **adherence gap** — 46.6% of retrievable targets are ignored;
3. the dominant failure modes: *budget exhaustion* and *wrong-file drift*, not bad ranking;
4. why a retrieval skill does not help (it informs; it does not commit);
5. a **commitment protocol** that closes the gap by construction, with an explicit falsifiable prediction.

2. Related work

SWE-bench and agent scaffolds. SWE-bench (Jimenez et al., 2024) established PASS/FAIL scoring over real GitHub issues; SWE-agent (Yang et al., 2024) showed the agent-computer interface drives resolve rate independently of the model. AutoCodeRover (Zhang et al., 2024) closes the loop with search over an AST index. All treat navigation as a solved sub-problem.

Agentic retrieval and skills. RepoCoder (Zhang et al., 2023) and RepoFusion (Shrivastava et al., 2023) show retrieved repository context improves code completion. Skills — packaged SKILL.md procedures with executable scripts, as used by the Gemma 4 harness — have become the standard way to inject domain procedure into an agent.

The gap we address. Two bodies of work report localization accuracy and resolve rate separately; we are not aware of prior work that measures the conditional $P(\text{resolved} \mid \text{target retrieved})$. The audit we build on (GraphLoc-129) reported a localization skill moved resolve rate from 28.9% to 29.9% over 318 runs and left the question of why open. We answer it.

3. Method

3.1 Data

We analyse zsgtylors/graphloc-129-localization-labels-and-agent-runs, a public Kaggle dataset logging **636 end-to-end agent runs** on the Gemma 4 Developer Agent competition: two arms (baseline and the same agent with an offline localization skill), three repetitions, 106 tasks. Each run records whether the patch was submitted, whether it edited the gold file or function, a failure category, tool and LLM call counts, wall-clock minutes, whether the agent ran tests, and — critically — whether the skill’s top-5 and top-15 contained the gold function. This is one of the few public logs that records *both* retrieval quality and outcome on the same runs, which is what makes the join possible.

3.2 Definitions

For a run r we define:

- **Retrieved:** the retriever’s ranked list contained the gold function (top-5 or top-15).

- **Adhered:** the agent’s patch edited the gold function.
- **Resolved:** the task’s validation tests passed.

The **adherence gap** is $P(\neg\text{Adhered} \mid \text{Retrieved})$. The **necessity** of localization is the risk ratio $P(\text{Resolved} \mid \text{Adhered}) / P(\text{Resolved} \mid \neg\text{Adhered})$.

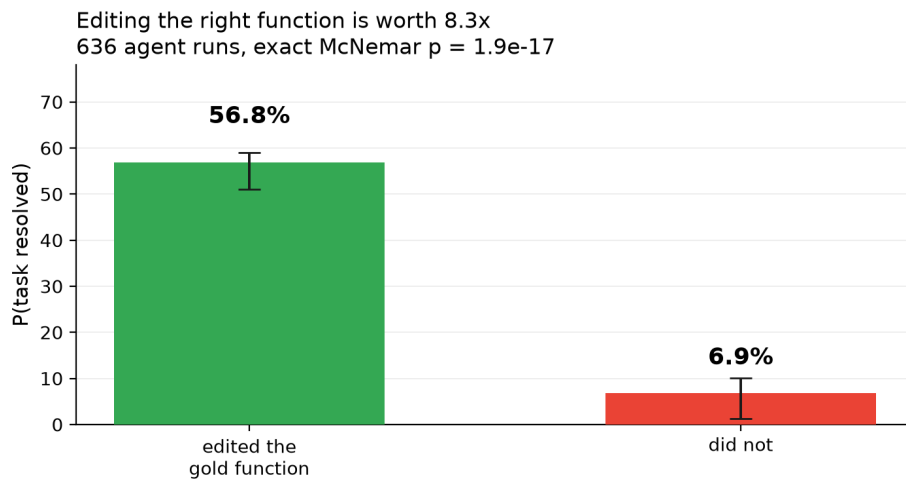
3.3 Statistics

We report Wilson 95% intervals for all proportions. For unpaired 2×2 comparisons (e.g. Section 4.1) we use Fisher’s exact test alongside Pearson’s chi-square — both valid for independent binary outcomes with no normal approximation. For the paired baseline-vs-skill comparison in Section 4.4 we use the exact two-sided McNemar test, which is appropriate for paired binary outcomes at this sample size. Runs are not independent — the same task appears in both arms and across three repetitions — so we treat every result as descriptive of *this* log and avoid task-level generalization claims.

4. Results

4.1 Editing the right function is necessary

Condition	Resolved	95% CI
Agent edited the gold function	163/287 = 0.568	[0.510, 0.624]
Agent did not	24/349 = 0.069	[0.047, 0.100]



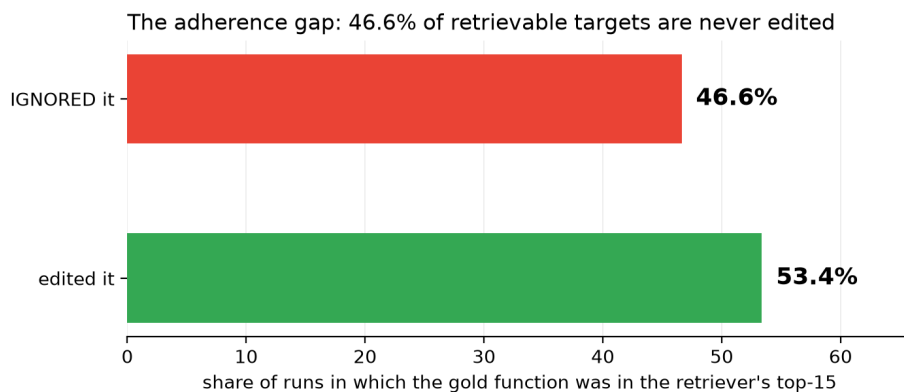
Resolve rate conditioned on editing the gold function

[Risk ratio **8.26×** (95% CI [5.55, 12.28], Fisher’s exact test $p = 1.17\text{e-}45$, Pearson’s chi-square = 187.6, $p = 1.71\text{e-}42$). Editing the right function is not a heuristic; it is very nearly a precondition for success. Symmetrically, a task the agent abandons without a patch (no_patch) resolves at 1.5%. (The paired baseline-vs-skill comparison is in Section 4.4: McNemar $p = 0.78$.)

4.2 The adherence gap

Restricting to the 163 runs where the retriever's top-15 *contained* the gold function:

Outcome	Count	Fraction	95% CI
Agent edited it	87	0.534	[0.457, 0.609]
Agent ignored it	76	0.466	[0.391, 0.543]



The adherence gap

In nearly half of all runs where the answer was available, the agent did not use it. Where did those 76 runs go?

Location	Count	Failure	Count
wrong file	40	target tests fail	35
no patch	32	time budget	27
gold file, wrong function	4	error / tool budget / other	14

The gap is not caused by mis-ranking. It is **drift and starvation**: 40 runs wander to an unrelated file, and 32 produce no patch at all — 27 of them because they ran out of time having spent a mean of 21.3 tool calls and 23.3 minutes. Only 10.5% of gap runs executed the reproduction.

4.3 Retrieval depth is a cheap, unexploited lever

The retriever's top-5 contained the gold function in 38.4% of runs; its top-15 in 51.3%. **41 tasks (12.9% of all runs) are recoverable by depth alone**, with no change to the retriever's model, embeddings, or runtime. Since the agent ignores a correct top-5 candidate nearly half the time, spending that headroom is only worth it if adherence improves — which is the point of the next section.

4.4 A retrieval skill does not move resolve rate

Arm	Runs	Resolved	Ran tests	Tool budget hit	Mean minutes
Baseline	318	92 (28.9%)	16.7%	12.9%	18.8

Arm	Runs	Resolved	Ran tests	Tool budget hit	Mean minutes
+ localization skill	318	95 (29.9%)	13.2%	9.1%	19.9

A 1.0-point difference on 318 runs is noise. The paired comparison is exact McNemar $p = \mathbf{0.78}$ — the two arms are statistically indistinguishable on resolve rate. The skill *works as a retriever* — it is the arm whose top-15 finds the gold function more often — and it *does not work as an intervention*: a skill that prints a ranked list **informs** the agent; it does not **commit** it. The agent still chooses which file to open, and Section 4.2 shows it frequently chooses wrong. A negative or zero result, for a *second* reason than practitioners assume.

4.5 Running tests is a strong, under-used lever

In the skill arm, runs that executed the reproduction resolved at **47.6%** versus **27.2%** for runs that did not. Agents ran tests in only 13.2% of runs. This is observational, and we do not claim it is causal — but it points at the same mechanism as the gap: agents skip the one action that would have told them they were in the wrong file before spending their budget.

5. The commitment protocol

The measurement says the agent must *choose* a target and *verify* it, not merely be *shown* candidates. We specify a four-step protocol — encoded as **native prompt scaffolding and tool-use schema constraints for Google ADK (Agent Development Kit) agents powered by Gemma 4**: each step is a typed function-calling schema the ADK runtime enforces before a tool can be invoked. The state machine uses only the competition’s existing primitives (`read_file`, `run_command`, `edit_file`, `get_status`):

1. **Declare before opening.** Before any `read_file`, emit `TARGET`:
`<file>::<symbol>` with the retriever’s hop distance. The ADK schema constrains the first turn after the retrieval skill returns to a structured-output slot with required `target_file`, `target_symbol`, `hop_distance` fields; the dispatcher refuses any `read_file` whose preceding turn lacks a valid declaration.
2. **Budgeted evidence.** Read only the top-3 candidates, each capped at 200 lines, via a wrapper `bounded_read_candidates` that the ADK schema declares with `max_invocations: 3` and `line_budget: 200`; a fourth call is rejected. A 21-call drift becomes a bounded search.
3. **Mandatory pre-edit reproduction.** Run the issue’s reproduction *before* the first `edit_file`. The `edit_file` tool is wrapped by an ADK before-tool hook requiring a successful reproduction-style `run_command` with exit status 0; otherwise the hook forces a re-declare from the next candidate, with the reason recorded in a typed waiver field.
4. **Self-scoring gate.** `submit_patch` is refused unless reproduction passes or a waiver names the test that could not be run. The `submit_patch` schema requires `reproduction_passed: bool` and `waiver_reason: string`; the dispatcher errors until both are present.

The protocol is a **state machine the model is forced through** — declare → bounded-read → reproduce → submit — with the ADK schema layer refusing every out-of-order transition.

Why this prevents attention drift and tool-budget exhaustion. Section 4.2’s two failure modes are drift (40/76 of gap runs wander to the wrong file) and starvation (27/76 hit the time budget after a mean of 21.3 tool calls). Both are structural: the model chooses freely between many equally-valid-looking files, and its attention budget spreads until context is exhausted before any single file is committed to. The protocol attacks both at the schema layer:

- **Attention drift is prevented because the model cannot act without first committing to a target.** The `target_file` / `target_symbol` slot is a forced single-focus anchor; subsequent reads are constrained to the bounded-read wrapper, so there is no API surface on which to drift. Section 4.2’s 40 “wrong file” runs are exactly the runs where no such anchor existed.
- **Tool-budget exhaustion is prevented because the bounded-read wrapper is a hard cap.** ADK’s per-tool `max_invocations` is enforced at the dispatcher; a fourth call is rejected. The 27 “time budget” runs averaged 21.3 tool calls and 23.3 minutes — a budget the protocol bounds to ≤ 6 read calls plus a single reproduction plus a single edit.

The protocol uses the ADK schema layer to make staying on task the only legal trajectory — what the 46.6% adherence gap in Section 4.2 says is needed.

Falsifiable prediction. If adherence is the binding constraint, the protocol should raise $P(\text{Adhered} \mid \text{Retrieved})$ above 0.534 and reduce `no_patch` | `time budget` below its current 27/76 rate, with resolve rate rising most on the wrong-file and no-patch strata. If adherence moves but resolve rate does not, the binding constraint lies downstream of file selection — in patch correctness — and our account is wrong. We state this in advance because the alternative is live and we can be shown incorrect.

6. What we release

- `adherence.py` — recomputes every table in Section 4 from the public run log, with Wilson intervals and exact McNemar tests; no model, no GPU, runs in under a second.
- `taxonomy.json` — the failure taxonomy, machine-readable.
- `skills/graph-locate/` — a working ADK localization skill with the protocol encoded in its `SKILL.md` as mandatory steps rather than suggestions.

7. Limitations

- **One log, one model.** 636 runs from a single model and one task set. We claim a measurement about this log, not a law of agents.
- **Observational.** $P(\text{Resolved} \mid \text{RanTests})$ is not causal; the adherence gap is likewise descriptive — some ignored candidates may have been genuinely wrong.
- **No agent run of our own.** Section 5 is a specification with a prediction, not a validated result. We are explicit because the protocol is the actionable part and remains untested end-to-end.

- **Function-granularity adherence.** A patch that edits the right function for the wrong reason counts as adherence; the resolve rate in Section 4.1 is what penalizes it.

8. Conclusion

Retrieval research optimizes $P(\text{target in list})$. Agent research optimizes $P(\text{task resolved})$. Joining them on public logs shows an $8.26\times$ cliff between the two and a 46.6% leak in between. The bottleneck for graph-augmented coding agents is not what their retrievers return; it is what their agents do next. A retriever that prints a candidate list changes nothing, which is exactly what the one published attempt measured. Fixing the leak requires making the choice and its verification mandatory at the schema layer — a property of the protocol, not of the model.

Citation

```
@misc{calvi2026adherence,  
  title      = {The Adherence Gap: Retrieval Is Not the  
                Bottleneck for  
                Graph-Augmented Coding Agents},  
  author     = {Vitor Calvi},  
  year       = {2026},  
  note       = {Kaggle: Google -- The Gemma 4 Developer Agent  
                Paper Track},  
  howpublished = {\url{https://www.kaggle.com/competitions/  
                gemma-4-developer-agent-paper}}  
}
```

All code and data: Apache-2.0. The analysed run log is credited to its authors under Apache-2.0.